

Kiadásfigyelő Webalkalmazás

MSSQL, ASP.NET CORE, REACT

Tárgy adatai:	Önálló Labor (BMEVIAUAL04)
Téma:	ASP.NET Core alapú webalkalmazás fejlesztése
Félév:	2025-2026 tavaszi félév
Konzulens:	Gincsei Gábor
Hallgató:	Bokor Ádám (UVMXFZ)

Tartalom

Bevezetés	2
Alkalmazott Technológiák	2
Architektúra & Tervezés	2
Architektúra	3
Tervezés	4
Entity Framework Core (ORM).....	4
Megvalósítás	6
Data Access Layer Projekt	6
Service Projekt.....	6
Commands	7
Services	7
Exceptions.....	9
Mappings.....	9
Data Transfer Objects Projekt	10
Server Projekt.....	11
ContextProvider	11
Handlers	11
Controllers.....	12
Tesztelés	13
Frontend	14
Felépítés	14
Irányítás.....	14
Kommunikáció.....	14
Összegzés.....	15

Bevezetés

Az előző félév folyamán két fontos tárggyal találkoztam: *Adatvezérelt rendszerek*, *Kliensoldali rendszerek*, amelyek tananyagából sok fontos dolgot tanultam, és segítettek megérteni a frontend és backend technológiák működését és sajátosságait. Az eddig elsajátított tudásom továbbfejlesztése érdekében az *Önálló Laboratórium* témájaként egy Full-Stack alkalmazás fejlesztését vállaltam. Az elmúlt 14 hét folyamán egy kiadásfigyelő webalkalmazást valósítottam meg.

A projekt célja számomra tehát a backend és frontend technológiákhoz kapcsolódó ismereteim bővítése volt. Az alkalmazás segítségével a felhasználó kezelheti a saját, illetve családtagjai tranzakcióit, számláit, amelyekről részletes információkat szerezhet meg a különböző diagramok, és listázási lehetőségek segítségével. A tranzakciókra szűrni lehet kategória, pénztárca és dátum szerint, de az egyes elemek módosíthatóak is, illetve lehetséges limitek hozzáadása is.

Az alábbiakban a fejlesztés fontosabb részleteit mutatom be kódrészletek és képernyőképek segítségével.

Alkalmazott Technológiák

A témakiírás alapvetően csak a backend platformját határozta meg, ami azt jelenti, hogy az adatbázist, illetve a frontendet megvalósító keretrendszert és technológiákat én választhattam.

- Backend: ASP.NET Core + Entity Framework Core (.NET 10)
- Adatbázis: Microsoft SQL Server [v17.0]
- Frontend: Vite (React) [v7.3.3]
- Tesztelés: JS Axios Package

Az adatbázis esetén a választásom egyértelmű volt, hiszen a Microsoft fejlesztésű ASP.NET Core és EF Core alapvetően a szintén általuk fejlesztett MSSQL adatbázissal működik a legkevesebb extra konfiguráció állítgatás nélkül. A frontend megvalósításához a Kliensoldali rendszerek tárgy folyamán is használt React és Vite technológiát választottam, mivel a tárgy háziját React-ot használva kellett megvalósítani, vagyis ebben volt már programozói tapasztalatom.

A fejlesztéshez használt környezetek, eszközök:

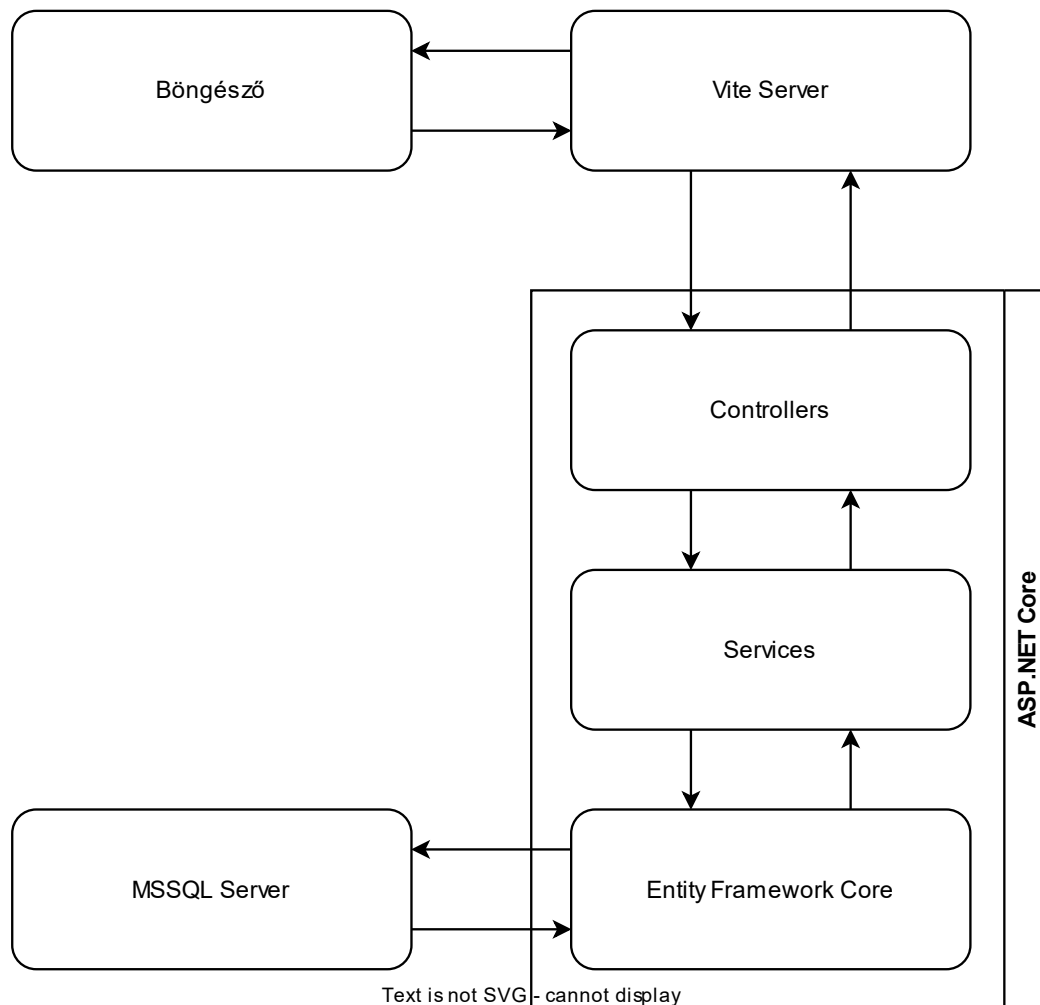
- Microsoft Visual Studio 2026 Community Edition
- Microsoft Visual Studio Code
- Microsoft SQL Server Management Studio 22
- JetBrains DataGrip

Architektúra & Tervezés

A félév során, a téma által kikötött backend fejlesztése volt számomra a prioritás, már csak azért is, mert egy rossz, vagy hiányos backend mellett nem lehetséges rugalmasan fejleszteni a frontendet. Emiatt a backend megfelelő tervezése elengedhetetlen volt ahhoz, hogy az alkalmazás elkészüljön a 14 hét folyamán.

Architektúra

Először az ideális architektúra kiválasztása kellett, hogy megtörténjen. Ezen nem gondolkoztam sokat, egyértelmű volt, hogy az iparban is elterjedt és leggyakrabban használt 3 rétegű architektúrát választottam. Ez azt jelenti, hogy a webalkalmazásnak 3 rétegből kell, hogy álljon. Adatelérési, Üzleti-logikai és Prezentációs réteg.



A fent látható ábra mutatja be az alkalmazás konkretizált felépítését. Amikor egy felhasználó kérést küld a webalkalmazás frontendjén (Vite Server-en) keresztül, az egy proxyn keresztül beérkezik a backendnek. Ezeket az API végpontokat az ASP.NET Core „mapolja” a Controllers-ben található osztályok és metódusok segítségével. Ezek meghívják a végponthoz tartozó Service osztály megfelelő metódusát, ami megvalósítja a lekérdezéshez és adatmanipulációhoz szükséges autorizációs logikát, illetve végrehajtja az adatbázis-lekérdezéseket vagy mentéseket az Entity Framework Core segítségével. Az adatok perzisztens tárolásáért, illetve néhány egyedhalmaz attribútumára definiált validációs kényszerért az MSSQL Server felel.

Tervezés

Regisztráció után minden felhasználó automatikusan bekerül egy családba, amiben egyedül ő szerepel. A család további felhasználók meghívásával bővíthető. Jogosultságokat csak a „családfők” oszthatnak. Mindig kell, hogy legyen legalább egy családfő, aki a család létrehozásakor a létrehozó. Az egyes családtagoknak költekezési limit is állítható. Erre csak a családfőknek van jogosultsága. A limit beállítható, illetve törölhető is. Költekezési limit állítható saját kiadásokra is. Az alkalmazásban tárcák vehetőek fel, melyeknek beállítható a típusa és kezdőösszege. A felhasználók és a tárcák közötti reláció több-több, tehát egy tárca akár több felhasználóhoz is tartozhat. Egy bevétel/kiadás nyomon követéséhez szükség van arra, hogy az bekerüljön az alkalmazásba. Ehhez a felhasználó létrehoz egy tranzakciót, aminek megadja az ahhoz tartozó tárcát, és a tranzakció nevét, leírását, kategóriáját és dátumát. Az egyes, már megadott tranzakciók adatai módosíthatóak, illetve törölhetőek. Az egyes tranzakciókhoz kategóriák is rendelhetőek. Az alkalmazás alapvető kategóriái mellé a felhasználó felvehet több, a családhoz tartozó kategóriát is. A kategóriákra szűrni is lehetséges.

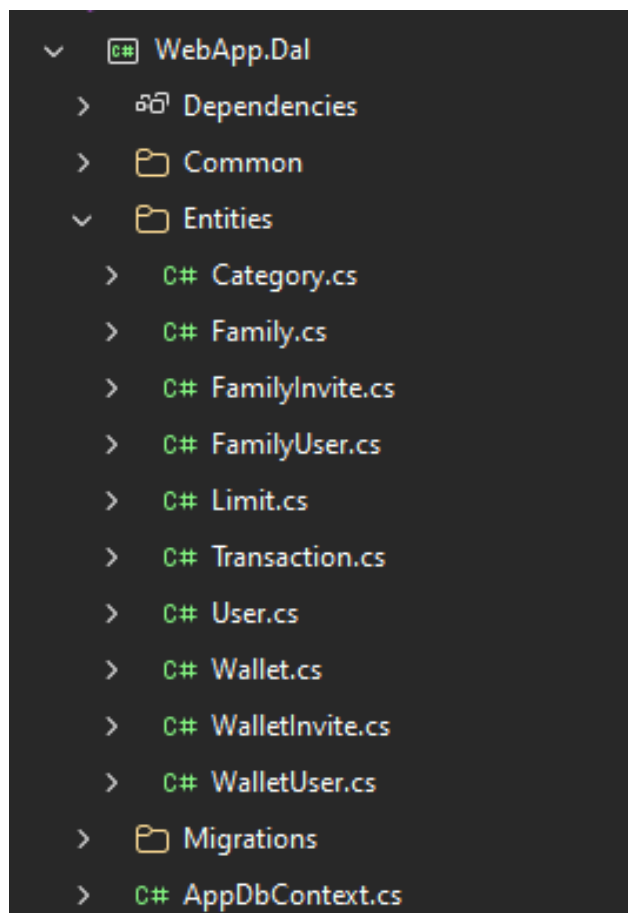
Entity Framework Core (ORM)

Az alkalmazás tervezése az adatbázis sémák definiálásával kezdődött. Az alapvető egyedhalmazok egyértelműek voltak, azonban később szükséges volt azok számának bővítése is. Ehhez az Entity Framework Core csomagot használtam, amivel automatikusan „mappolható” egy ASP.NET Core által használt objektum az adatbázisba (ORM).

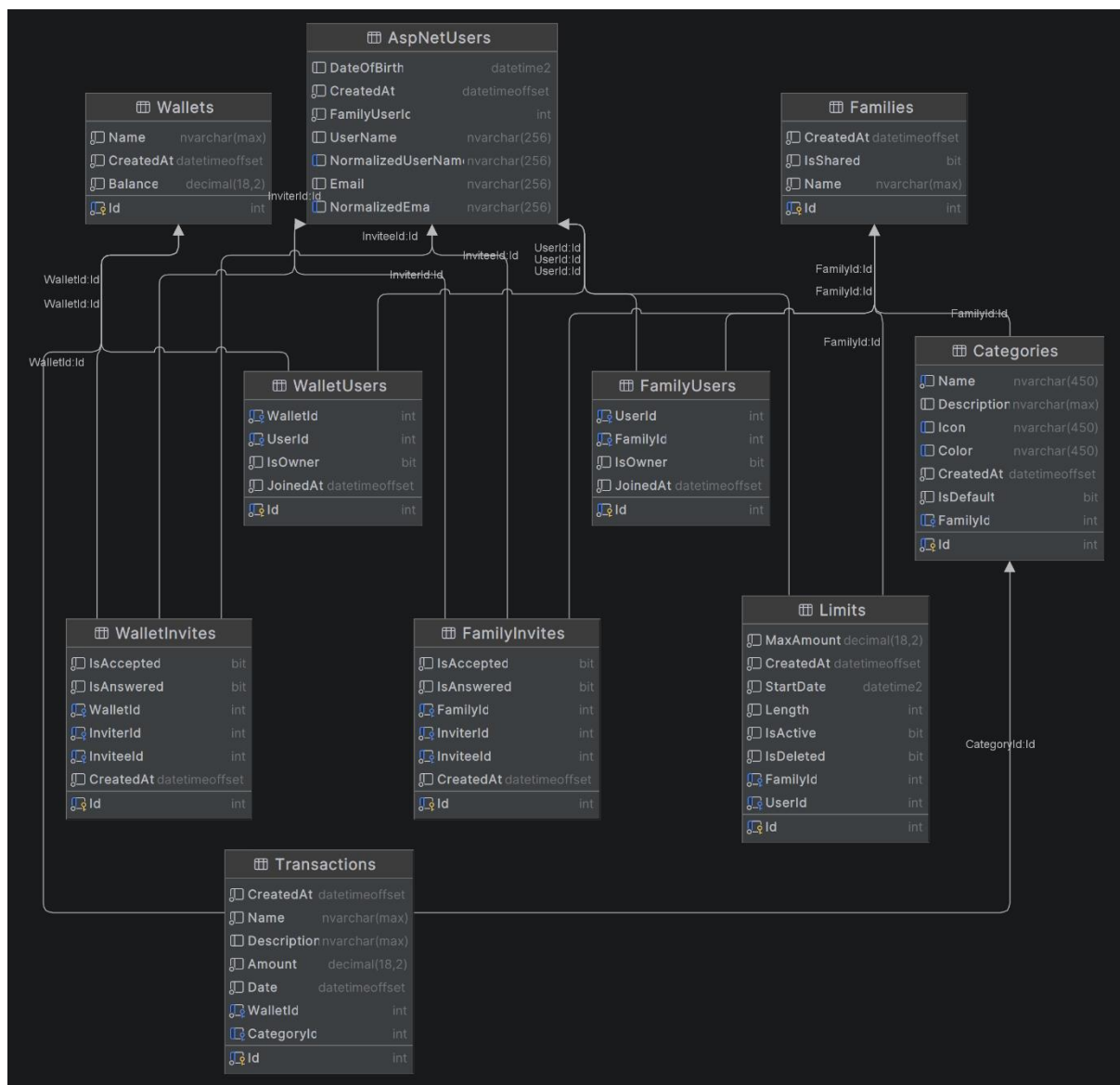
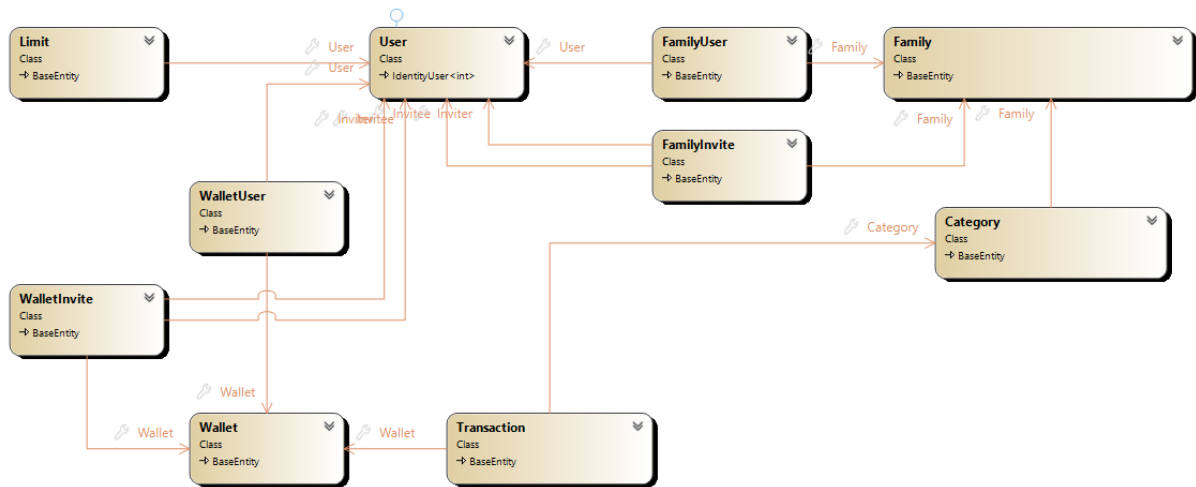
Az alapvető egyedhalmazok, és azok szerepe:

Family:	Egy családot...
User:	Egy felhasználót...
Wallet:	Egy számlát, tárcát, megtakarítást...
Transaction:	Egy tranzakciót...
Category:	Egy kategóriát...
Limit:	Egy limitet...
FamilyInvite:	Egy családhoz tartozó meghívót...
WalletInvite:	Egy wallet-hez tartozó meghívót...
FamilyUser:	Egy felhasználó és családja közötti kapcsolatot...
WalletUser:	Egy felhasználó és egy wallet közötti kapcsolatot reprezentál

Az egyes Invite objektumok felelnek egy-egy meghívóért, amit a küldő küld egy címzettnek, amikor az be akarja hívni a másik felhasználót a családjába, vagy a tárcájába. Az egyes User objektumok pedig valójában kapcsoló táblák egy-egy User és Family, vagy Wallet között, aminek segítségével további tulajdonságok adhatók meg



a kapcsolathoz, például a felhasználó szerepe az adott családhoz vagy tárcához (Owner, Member). Az egyedhalmazok közötti kapcsolatokat az alábbi képek tartalmazzák:



Megvalósítás

Az alkalmazás gyakorlati megvalósítási szakasza a backend projekt létrehozásával kezdődött. Ehhez a Visual Studio által nyújtott template-k közül az „ASP.NET Core WEB Api” opciót választottam.

Ez a template alapvetően egy projektet hoz csak létre, azonban a teljes backend több, egymástól funkcióban eltérő és különálló részekből áll. Hogy ezek strukturálisan is szeparálva legyenek egymástól, további 3 projekt került létrehozásra a Solution-on belül, amelyek neve és szerepköre rendre:

- WebApp.Dal
 - o Az adatbázis és az EF Core által használt objektumokat tartalmazó projekt
- WebApp.Service
 - o Az adatbázis üzleti-logika-szerinti módosítását végző Service-k és segédosztályokat tartalmazó projekt
- WebApp.Server
 - o A REST API végpontokat definiáló Controller-eket, és a teljes backend belépési pontját tartalmazó projekt
- WebApp.Dto
 - o A Service-k által kiküldött, az interneten utazó kérések és válaszok formáját meghatározó rekordokat tartalmazó projekt.

Data Access Layer Projekt

A Data Access Layer projekt struktúrája a következő:

- Common
 - o Minden EF Core entitás által megosztott attribútumokat tartalmazó Interfészt és absztrakt ösztályt tartalmazó névtér
- Entities
 - o Az összes, backend által használt ORM objektumot tartalmazó névtér
- ApplicationDbContext
 - o Adatbáziskapcsolat kezeléséért,
 - o Táblák definiálásáért,
 - o Alap adatok betöltéséért felelős osztály

Service Projekt

Talán a backend legfontosabb osztályait tartalmazó projekt. Itt fontos volt számomra, hogy szeparálva legyenek a tényleges üzleti-logikát megvalósító osztályok a segédosztályoktól.

A projekt struktúrájának fontosabb elemei:

- Commands
- Exceptions
- Mappings
- Services

Commands

Ebben a névtérben található az egyes Service-metódusok meghívásához szükséges adatok sémáját definiáló rekordok. Például:

```
5 references
public record GetAllTransactionsCommand
{
    2 references
    → required public int InvokerId { get; init; }
    6 references
    → public IEnumerable<int>? CategoryIds { get; init; }
    4 references
    → public IEnumerable<int>? WalletIds { get; init; }
    3 references
    → public DateTime? StartDate { get; init; }
    3 references
    → public DateTime? EndDate { get; init; }
}
```

Ez a rekord a TransactionService GetAllTransactionsAsync metódus futtatásához szükséges adatok sémáját definiálja. Az InvokerId minden rekord része, ez azonosítja a metódust meghívó felhasználót.

Services

Az összes Service megvalósítását tartalmazó névtér. Egy Service osztály általában a következő paraméterek kapja, amikkel dolgozik:

```
1 reference
public class TransactionService(
    → AppDbContext db,
    → IMapper mapper,
    → IValidationService validationService,
    → ICommonService commonService
) : ITransactionService
{
    → private readonly AppDbContext db = db;
    → private readonly IMapper mapper = mapper;
    → private readonly ICommonService commonService = commonService;
    → private readonly IValidationService validationService = validationService;
    |
    2 references
}
```

Az egyes Service-k gyakran hivatkoznak egy ún. CommonService osztályra, amiben a gyakran megismételt adatbázis lekérdezések találhatóak. A ValidationService osztályban található metódusok felelnek az egyes entitások üzleti-logika-szerinti validációjáért, például, hogy egy tárcán található pénzösszeg ne lehessen negatív értékű.

Példa egy Service metódusra:

```
2 references
public async Task<TransactionDataDto> CreateTransactionAsync(CreateTransactionCommand command, CancellationToken cancellationTo
{
    + //NOT FOUND
    + var wallet =
    + + await commonService
    + + .GetWalletEntityAsync(
    + + + command.WalletId,
    + + + loadUsers: true,
    + + + cancellationToken: cancellationToken
    + + );
    + var invoker =
    + + await commonService
    + + .GetUserEntityAsync(
    + + + command.InvokerId,
    + + + loadFamily: true,
    + + + cancellationToken: cancellationToken
    + + );
    + //AUTHORIZATION
    + var isInvokerInWallet =
    + + await db.WalletUsers.AnyAsync(wu => wu.WalletId == command.WalletId && wu.UserId == invoker.Id, cancellationToken);
    + if(!isInvokerInWallet)
    + {
    + + throw new ForbiddenException(Messages.FailedToCreate("Transaction"), $"You '{invoker.Email}' are not a member of the wa
    + + );
    + }
    + //VALIDATION & CREATE
    + var id = await commonService.CreateTransactionAsync(command, cancellationToken);
    + var transaction = await commonService.GetTransactionEntityAsync(id, loadWallet: true, cancellationToken);
    + return mapper.Map<TransactionDataDto>(transaction);
}
```

Minden Service metódus a *command*-ban küldött adatok valóságát vizsgálja először, hiszen, ha a megadott tárca, amire a tranzakciót létre akarjuk hozni nem létezik, **akkor értelmetlen az egész kérés**. A metódusok ezen részének a *NOT FOUND* nevet adtam.

Ezután az adatbázisból lekérdezzük a kérést létrehozó felhasználóhoz tartozó User objektumot, így meg tudjuk nézni, hogy az adott felhasználó tagja-e az adott tárcának, ugyanis, ha nem, **akkor értelmetlen az egész kérés**, hiszen egy tárcához hozzá nem férő felhasználónak nem szabad hagyni, hogy tranzakciót hozzon létre az adott tárcán. A metódusok ezen részének a *AUTHORIZATION* nevet adtam.

Ha tudjuk, hogy léteznek a megadott elemek, és a felhasználó hozzá is férhet, akkor jöhet a validáció, illetve a **CREATE/READ/UPDATE/DELETE** művelet az adatbázison.

Exceptions

Ez a névtér tartalmazza a Service-k által dobott kivételeket. Tartalma a következő:

```
namespace WebApp.Service.Exceptions;

7 references
public abstract class ApiException(string statusCodeTitle, int statusCode, string exceptionMsg, string reasonMsg = "") : Exception(exceptionMsg + reasonMsg)
{
    3 references
    public int StatusCode { get; } = statusCode;
    1 reference
    public string StatusCodeTitle { get; } = statusCodeTitle;
    1 reference
    public string ExceptionMessage { get; } = exceptionMsg;
    1 reference
    public string ReasonMessage { get; } = reasonMsg;
}

33 references
public class BadRequestException(string exceptionMsg, string reasonMsg)
{
    : ApiException("Bad Request", StatusCodes.Status400BadRequest, exceptionMsg, reasonMsg)
}

1 reference
public class UnauthorizedException(string exceptionMsg, string reasonMsg)
{
    : ApiException("Unauthorized", StatusCodes.Status401Unauthorized, exceptionMsg, reasonMsg)
}

44 references
public class ForbiddenException(string exceptionMsg, string reasonMsg)
{
    : ApiException("Forbidden", StatusCodes.Status403Forbidden, exceptionMsg, reasonMsg)
}

1 reference
public class NotFoundException(string exceptionMsg, string reasonMsg)
{
    : ApiException("Not Found", StatusCodes.Status404NotFound, exceptionMsg, reasonMsg)
}

5 references
public class ConflictException(string exceptionMsg, string reasonMsg)
{
    : ApiException("Conflict", StatusCodes.Status409Conflict, exceptionMsg, reasonMsg)
}

0 references
public class InternalServerErrorException(string exceptionMsg, string reasonMsg)
{
    : ApiException("Internal Server Error", StatusCodes.Status500InternalServerError, exceptionMsg, reasonMsg)
}
```

Az egyes kivételek mind egy-egy 40X-es HTTP Status-nak felelnek meg. A Service osztályok metódusai (beleértve a ValidationService-t és a CommonService-t is) ezeket a kivételeket dobják a kérés teljesíthetetlensége szerint.

Mappings

Ahhoz, hogy az egyes DB entitásokat ne kelljen kézzel mindig DTO rekordokká alakítani, felvettem a projekthez az AutoMapper csomagot. Ez egy regisztrációhoz kötött, licensz-alapú mapper, ami segítségével könnyen megadható, hogyan történjen egy entitás DTO-vá alakítása. Példa egy Mapper osztályra:

```
2 references
public class InviteMappingProfile : Profile
{
    0 references
    public InviteMappingProfile()
    {
        CreateMap<FamilyInvite, FamilyInviteDto>()
        {
            .ForMember(dest => dest.Id, opt => opt.MapFrom(src => src.Id))
            .ForMember(dest => dest.Family, opt => opt.MapFrom(src => src.Family))
            .ForMember(dest => dest.InviterUser, opt => opt.MapFrom(src => src.Inviter))
            .ForMember(dest => dest.InvitedUser, opt => opt.MapFrom(src => src.Invitee))
            .ForMember(dest => dest.IsAccepted, opt => opt.MapFrom(src => src.IsAccepted))
            .ForMember(dest => dest.CreatedAt, opt => opt.MapFrom(src => src.CreatedAt))
        };
        CreateMap<WalletInvite, WalletInviteDto>()
        {
            .ForMember(dest => dest.Id, opt => opt.MapFrom(src => src.Id))
            .ForMember(dest => dest.Wallet, opt => opt.MapFrom(src => src.Wallet))
            .ForMember(dest => dest.InviterUser, opt => opt.MapFrom(src => src.Inviter))
            .ForMember(dest => dest.InvitedUser, opt => opt.MapFrom(src => src.Invitee))
            .ForMember(dest => dest.IsAccepted, opt => opt.MapFrom(src => src.IsAccepted))
            .ForMember(dest => dest.CreatedAt, opt => opt.MapFrom(src => src.CreatedAt))
        };
    }
}
```

Data Transfer Objects Projekt

Ez a projekt tartalmazza az összes DTO-t, amely használ a backend. A DTO-k két részre bonthatóak:

- Request
- Response

Ehhez létrehoztam egy jelölő-attribútumot, ami nem csinál semmit, csak jelöli a könnyű megkülönböztethetőségért, hogy melyik DTO érkező, és melyik kimenő.

Példa pár DTO-hoz:

```
[Request]
1 reference
public record CreateLimitDto
{
    1 reference
    → public int UserId { get; init; }
    1 reference
    → public decimal MaxAmount { get; init; }
    1 reference
    → public DateOnly StartDate { get; init; }
    1 reference
    → public DateOnly EndDate { get; init; }
    1 reference
    → public bool IncludeIncome { get; init; }
    1 reference
    → public bool IsActive { get; init; }
    1 reference
    → public string? Description { get; init; }
};

[Request]
0 references
public record GetLimitByIdDto
{
    0 references
    → public int Id { get; init; }
};

[Request]
1 reference
public record UpdateLimitDto
{
    1 reference
    → public decimal? MaxAmount { get; init; }
    1 reference
    → public string? Description { get; init; }
    1 reference
    → public DateOnly? StartDate { get; init; }
    1 reference
    → public DateOnly? EndDate { get; init; }
    1 reference
    → public bool? IsActive { get; init; }
    0 references
    → public bool? IncludeIncome { get; init; }
};
```

Server Projekt

Ebben a projektben található a backend szerver belépési pontja. Itt található az összes API végpontot definiáló Controller osztály, az általam készített kivételeket elkapó Handlerek, illetve a HTTP kérésből, a kérést végző felhasználó azonosításáért felelős ContextProvider is.

ContextProvider

Ez az osztály tudja egy HTTP kéréshez kikeresni a kérést készítő felhasználó azonosítóját.

```
14 references
public interface IRequestContextProvider
{
    49 references
    public int InvokerId { get; }
}

1 reference
public class RequestContextProvider(IHttpContextAccessor httpContextAccessor) : IRequestContextProvider
{
    private readonly IHttpContextAccessor httpContextAccessor = httpContextAccessor;

    49 references
    public int InvokerId
    {
        get
        {
            return int.TryParse(
                httpContextAccessor.HttpContext?.User.FindFirstValue(ClaimTypes.NameIdentifier), out var userId)
                ? userId : throw new UnauthorizedException("Failed to retrieve user information!", "User is not authenticated!");
        }
    }
}
```

Handlers

Két Handler van definiálva és regisztrálva az alkalmazásban:

- ApiExceptionHandler
 - o Elkapja azokat a kivételeket, amelyeknek típusa az általam létrehozott, a Service projektben található ApiException.
- GeneralExceptionHandler
 - o Elkap minden más kivételt, hogy az alkalmazás ne „szálljon” el.

A Handler-ek használata nem csak kivételek elkapására, és egy Error üzenet kiírására alkalmasak: segítségükkel megoldható, hogy bármilyen kivétel esetén a szerver egy közös interfészre illeszkedő objektumot küldjön a kérést végrehajtó végpontnak. ASP.NET Core alapú szerver esetén ezt a ProblemDetails osztály tudja nyújtani, amelyet külön regisztrálni kell a Program osztályban. Alább látható példa egy ExceptionHandler-re:

```
3 references
public class ApiExceptionHandler(ILogger<ApiExceptionHandler> logger) : IExceptionHandler
{
    private readonly ILogger<ApiExceptionHandler> logger = logger;

    0 references
    public async ValueTask<bool> TryHandleAsync(HttpContext httpContext, Exception exception, CancellationToken cancellationToken)
    {
        if(exception is not ApiException apiException) return false;

        httpContext.Response.StatusCode = apiException.StatusCode;

        var problemDetails = new ProblemDetails
        {
            Type = $"https://httpstatuses.com/{apiException.StatusCode}",
            Title = apiException.StatusCodeTitle,
            Status = apiException.StatusCode,
            Detail = apiException.ExceptionMessage,
            Instance = httpContext.Request.Path,
            Extensions =
            {
                ["reason"] = apiException.ReasonMessage
            }
        };

        await httpContext.Response.WriteAsJsonAsync(problemDetails, cancellationToken);
        logger.LogInformation(exception, "An API exception occurred and got caught: {Message}", exception.Message);
        return true;
    }
}
```

Controllers

Az összes Controller ebben a névtérben található. A Controllerek, a Servicekhez hasonlóan entitásszinten vannak szeparálva, tehát külön-külön létezik egy WalletController és például egy CategoryController.

A legtöbb Controller osztály alapvetően a következő paraméterekkel gazdálkodik:

```
9 [ApiController]
10 [Route("[controller]")]
11 [Authorize]
12 0 references
13 public class CategoryController(
14     > ICategoryService categoryService,
15     > IRequestContextProvider requestContextProvider
16     > ) : ControllerBase
17 {
18     > private readonly ICategoryService categoryService = categoryService;
19     > private readonly IRequestContextProvider requestContextProvider = requestContextProvider;
20 }
```

Attribútumok:

- ApiController
 - o Automatikusan modell validálás
- Route...
 - o Az adott Controlleren belüli végpontok előtti URL rész megadása, ami mindenhol a Controllerhez tartozó entitás neve.
- Authorize
 - o Automatikusan 403: Forbidden státusszal tér vissza az API hívás a Controlleren belüli végpontokra, ha a felhasználó nem hitelesíti magát

Paraméterek:

- Controllerhez tartozó Service
- ContextProvider

Egy-egy Controllerhez tartozó metódus felépítése mindig a következő:

1. A meghívandó Service metódushoz tartozó Command létrehozása
2. A Service metódusának meghívása
3. A válaszul kapott DTO objektum válaszkénti küldése

Példa egy API végponthoz:

```
[HttpPost]
0 references
public async Task<ActionResult<IEnumerable<TransactionDataDto>>> GetAllTransactionsAsync(
    > GetAllTransactionsDto request,
    > CancellationToken cancellationToken = default
    > )
{
    > var command =
    >     > new GetAllTransactionsCommand
    >     > {
    >         > InvokerId = contextProvider.InvokerId,
    >         > WalletIds = request.WalletIds,
    >         > CategoryIds = request.CategoryIds,
    >         > StartDate = request.startDate,
    >         > EndDate = request.endDate
    >     > };
    > var transactions = await transactionService.GetAllTransactionsAsync(command, cancellationToken);
    > return Ok(transactions);
}
```

Tesztelés

A kliensoldali projekt megvalósítása előtt szükséges volt a backend logika, és funkciók tesztelése. Ehhez az Axios JavaScript alapú csomagot használtam, amivel teszteseteket írtam az egyes végpontokhoz, illetve a komplexebb logikai működésekhez is. Példa egy tesztelésre:

```
describe('Family Invite API Tests', () => {
  beforeAll(async () => {
    await registerUser(emails[0], passwords[0])
    await registerUser(emails[1], passwords[1])
    headerWithCookie = await loginUser(emails[0], passwords[0])
  })

  test('Create Invite - User does not exist', async () => {
    const response = await api.post('family/invite/send', { userEmail: 'doesnotexist@doesnotexist.com' }, headerWithCookie )
    logResponse(response)
    expect(response.status).toBe(404)
  })

  test('Create Invite - Invalid Email Format', async () => {
    const response = await api.post('family/invite/send', { userEmail: 'this-is-not-a-valid-email.com' }, headerWithCookie )
    logResponse('Create Invite - Invalid Email Format', response)
    expect(response.status).toBe(400)
  })

  test('Create Invite - Success', async () => {
    const response = await api.post('family/invite/send', { userEmail: emails[1] }, headerWithCookie )
    logResponse(response)
    expect(response.status).toBe(200)
  })

  test('Create Invite - Already Sent', async () => {
    const response = await api.post('family/invite/send', { userEmail: emails[1] }, headerWithCookie )
    logResponse(response)
    expect(response.status).toBe(409)
  })
})
```

A tesztelés segítségével megbizonyosodtam a backend megfelelő működéséről, azonban később pár módosítás miatt a tesztesetek változó mennyiségét szintén módosítani kellett.

Frontend

A sikeres tesztelés után nekiálltam a frontend elkészítéséhez. A gyors építéshez React-ot használtam, esztétikus komponensekhez a shadcn-t könyvtárat, a diagramokhoz pedig ChartJS-t.

Felépítés

A frontend komponensalapú felépítése Element, és Page típusú komponensekből áll: az element-komponensek felelnek egy-egy gyakran használt, önállóan is értelmezhető elemekért, például egy listaért, vagy dialógusért, és ezekből épül fel egy-egy oldal, vagyis Page.

Irányítás

A frontenden történő irányítást, tehát az egyes route-ok és Page komponensek logikai összekötését a React *react-router-dom* modult használtam. Ennek segítségével könnyen „mapolható”, hogy melyik route-ra történő HTTP hívás esetén melyik Page jelenjen meg. A modul segítségével azt is megoldottam, hogy nem létező route-ok esetén egy együttes, *NotFound Page* jelenjen meg, ha pedig a felhasználó nincs autentikálva, vagyis bejelentkezve, akkor mindenképp csak a *Login Page* jelenjen meg neki.

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <RootPage />,
    errorElement: <NotFoundPage />
  },
  {
    hydrateFallbackElement: <SpinnerFallback />,
    children: [
      { path: 'login', element: <LoginPage /> },
      { path: 'signup', element: <SignupPage /> }
    ]
  },
  {
    element: <MainLayout />,
    loader: authLoader,
    hydrateFallbackElement: <SpinnerFallback />,
    children: [
      { path: 'dashboard', element: <DashboardPage /> },
      { path: 'family', element: <FamilyPage /> },
      { path: 'wallets', element: <WalletsPage /> },
      { path: 'transactions', element: <TransactionsPage /> },
      { path: 'categories', element: <CategoriesPage /> },
      { path: 'limits', element: <LimitsPage /> },
      { path: 'profile', element: <ProfilePage /> },
      { path: 'family/user/:familyUserId', element: <FamilyUserItemPage /> },
      { path: 'wallets/:walletId', element: <WalletItemPage /> },
      { path: 'wallets/user/:walletUserId', element: <WalletUserItemPage /> },
    ]
  }
])

export default function App() {
  return (<RouterProvider router={router} />)
}
```

Kommunikáció

A backend-el való összekötéshez, vagyis a kommunikáció gyors és automatikus megvalósításáért az NSwag csomagot használtam, amivel a DTO-k és API kérések, mint metódusok is automatikusan átvihetők a frontendre. Sajnos a kérések „mapolását” nem tudtam jól használhatóan megoldani, ezért az API hívások frontendes metódusait kézzel kellett megírni, ami azonban nem lassította a fejlesztést, bár tény, hogy nagyobb alkalmazásokhoz, ahol több API végpont is található, érdekesebb a hívásokat is automatikusan a frontendre vinni.

Összegzés

Az elmúlt 14 hét folyamán a fentebb olvasható fejezetekben tárgyalt tervezést és implementálást, és folyamatos tesztelést valósítottam meg. A félév folyamán nagyon sok új dolgot tanultam, elsősorban az ASP.NET Core keretrendszerrel kapcsolatban, de például esztétikus komponens könyvtárat is most használtam először.

Összesítve elégedett vagyok a munkámmal, egy olyan webalkalmazást hoztam létre, amit én is szívesen használnék, és lehet is hogy fogok, azonban pár lényeges QoL (Quality of Life) funkciót még mindenképp kívánatos lenne implementálni:

- Több és részletesebb diagram
- Backup és Restore megoldások
- Felhős integráció (szinkronizáció)
- Stb...